

Homework 3

https://github.com/hhafiya/software_architecture.git, branch **micro_hazelcast**

1. Запустити три екземпляра **logging-service**.

●	logging-service-2-1	e6e073ca1e35	software_architecture
●	hz-node-3-1	21dc2b72fe2e	hazelcast/hazelcast : 5703:5701
●	logging-service-1-1	1857ea9be293	software_architecture
●	logging-service-3-1	d14d6b402331	software_architecture

2. Запустити кластер Hazelcast з трьох серверів.

●	hz-node-2-1	cf41589cfbc3	hazelcast/hazelcast : 5702:5701	1.01%	2 minutes ago
●	hz-node-1-1	6aa17d443ebe	hazelcast/hazelcast : 5701:5701	1.06%	2 minutes ago
●	hz-node-3-1	21dc2b72fe2e	hazelcast/hazelcast : 5703:5701	0.94%	2 minutes ago

3. Записати 10 транзакцій msg1-msg10 через facade-service.

<pre>curl -X 'POST' \ 'http://localhost:8080/transaction' \ -H 'accept: application/json' \ -H 'Content-Type: application/json' \ -d '{ "user_id": "1234", "amount": 25.5 }' {"transaction_id": "1774013272679", "balance": 25.5}</pre>	<pre>curl -X 'POST' \ 'http://localhost:8080/transaction' \ -H 'accept: application/json' \ -H 'Content-Type: application/json' \ -d '{ "user_id": "1234", "amount": 35.5 }' {"transaction_id": "1774013298945", "balance": 61}</pre>	<pre>curl -X 'POST' \ 'http://localhost:8080/transaction' \ -H 'accept: application/json' \ -H 'Content-Type: application/json' \ -d '{ "user_id": "1234", "amount": 3.5 }' {"transaction_id": "1774013332487", "balance": 64.5}</pre>	<pre>curl -X 'POST' \ 'http://localhost:8080/transaction' \ -H 'accept: application/json' \ -H 'Content-Type: application/json' \ -d '{ "user_id": "1234", "amount": 3.5 }' {"transaction_id": "1774013361038", "balance": 68}</pre>	<pre>curl -X 'POST' \ 'http://localhost:8080/transaction' \ -H 'accept: application/json' \ -H 'Content-Type: application/json' \ -d '{ "user_id": "1234", "amount": 11.3 }' {"transaction_id": "1774013390815", "balance": 79.3}</pre>
<pre>curl -X 'POST' \ 'http://localhost:8080/transaction' \ -H 'accept: application/json' \ -H 'Content-Type: application/json' \ -d '{ "user_id": "1234", "amount": 21.3 }' {"transaction_id": "1774013428524", "balance": 100.6}</pre>	<pre>curl -X 'POST' \ 'http://localhost:8080/transaction' \ -H 'accept: application/json' \ -H 'Content-Type: application/json' \ -d '{ "user_id": "1234", "amount": 111.1 }' {"transaction_id": "1774013459684", "balance": 211.7}</pre>	<pre>curl -X 'POST' \ 'http://localhost:8080/transaction' \ -H 'accept: application/json' \ -H 'Content-Type: application/json' \ -d '{ "user_id": "1234", "amount": 0.1 }' {"transaction_id": "1774013486216", "balance": 211.8}</pre>	<pre>curl -X 'POST' \ 'http://localhost:8080/transaction' \ -H 'accept: application/json' \ -H 'Content-Type: application/json' \ -d '{ "user_id": "1234", "amount": 10.2 }' {"transaction_id": "1774013520223", "balance": 222}</pre>	<pre>curl -X 'POST' \ 'http://localhost:8080/transaction' \ -H 'accept: application/json' \ -H 'Content-Type: application/json' \ -d '{ "user_id": "1234", "amount": 10 }' {"transaction_id": "1774013547585", "balance": 232}</pre>

 **software_architecture-logging-service-1-1**
33b4417041fb [🔗](#) [software_architecture-logging-service-1:latest](#)

Logs

Inspect

Bind mounts

Exec

Files

Stats

```
INFO: Started server process [10]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
LOG: Connected to Hazelcast cluster
LOG: Received transaction 1774013272679
INFO: 172.18.0.11:54700 - "POST /log HTTP/1.1" 200 OK
LOG: Received transaction 1774013390815
INFO: 172.18.0.11:54982 - "POST /log HTTP/1.1" 200 OK
```

 **software_architecture-logging-service-2-1**
79716847de22 [🔗](#) [software_architecture-logging-service-2:latest](#)

Logs

Inspect

Bind mounts

Exec

Files

Stats

```
INFO: Started server process [10]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
LOG: Connected to Hazelcast cluster
LOG: Received transaction 1774013361038
INFO: 172.18.0.11:57438 - "POST /log HTTP/1.1" 200 OK
LOG: Received transaction 1774013428524
INFO: 172.18.0.11:48522 - "POST /log HTTP/1.1" 200 OK
LOG: Received transaction 1774013486216
INFO: 172.18.0.11:50198 - "POST /log HTTP/1.1" 200 OK
LOG: Received transaction 1774013547585
INFO: 172.18.0.11:58116 - "POST /log HTTP/1.1" 200 OK
```

Logs	Inspect	Bind mounts	Exec	Files	Stats
<pre>INFO: Started server process [10] INFO: Waiting for application startup. INFO: Application startup complete. INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit) LOG: Connected to Hazelcast cluster LOG: Received transaction 1774013298945 INFO: 172.18.0.11:52564 - "POST /log HTTP/1.1" 200 OK LOG: Received transaction 1774013332487 INFO: 172.18.0.11:49706 - "POST /log HTTP/1.1" 200 OK LOG: Received transaction 1774013459684 INFO: 172.18.0.11:52386 - "POST /log HTTP/1.1" 200 OK LOG: Received transaction 1774013520223 INFO: 172.18.0.11:55142 - "POST /log HTTP/1.1" 200 OK</pre>					

Отже, logging-service-1 отримав 2 транзакції, logging-service-2 отримав 4 і logging-service-3 теж 4.

4. Прочитати транзакції з facade-service

```
curl -X 'GET' \
'http://localhost:8080/user/1234' \
-H 'accept: application/json'
```

Logs	Inspect	Bind mounts	Exec	Files	Stats
<pre>INFO: Started server process [10] INFO: Waiting for application startup. INFO: Application startup complete. INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit) LOG: Connected to Hazelcast cluster LOG: Received transaction 1774013298945 INFO: 172.18.0.11:52564 - "POST /log HTTP/1.1" 200 OK LOG: Received transaction 1774013332487 INFO: 172.18.0.11:49706 - "POST /log HTTP/1.1" 200 OK LOG: Received transaction 1774013459684 INFO: 172.18.0.11:52386 - "POST /log HTTP/1.1" 200 OK LOG: Received transaction 1774013520223 INFO: 172.18.0.11:55142 - "POST /log HTTP/1.1" 200 OK INFO: 172.18.0.11:59168 - "GET /logs/1234 HTTP/1.1" 200 OK</pre>					

```
{
  "balance": 202,
  "transactions": [
    {
      "transaction_id": "1774013332487",
      "user_id": "1234",
      "amount": 0.5
    },
    {
      "transaction_id": "1774013459684",
      "user_id": "1234",
      "amount": 10.5
    },
    {
      "transaction_id": "1774013420524",
      "user_id": "1234",
      "amount": 21.5
    },
    {
      "transaction_id": "1774013363837",
      "user_id": "1234",
      "amount": 3.5
    },
    {
      "transaction_id": "1774013394015",
      "user_id": "1234",
      "amount": 11.5
    },
    {
      "transaction_id": "1774013320227",
      "user_id": "1234",
      "amount": 10.5
    },
    {
      "transaction_id": "1774013345705",
      "user_id": "1234",
      "amount": 10
    },
    {
      "transaction_id": "1774013298945",
      "user_id": "1234",
      "amount": 10.5
    },
    {
      "transaction_id": "1774013402216",
      "user_id": "1234",
      "amount": 0.5
    },
    {
      "transaction_id": "1774013272079",
      "user_id": "1234",
      "amount": 25.5
    }
  ]
}
```

logging-service-3 отримав запит на GET.

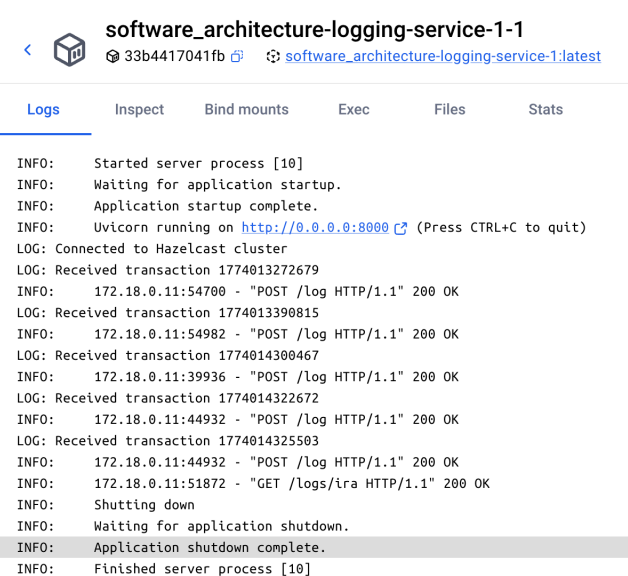
5. Перевірка відмовостійкості

- **Тест 1.** Вимкнути один/два екземпляри **logging-service** та перевірити чи зможемо записати та прочитати транзакції
docker stop software_architecture-logging-service-3-1

Logs	Inspect	Bind mounts	Exec	Files	Stats
<pre>INFO: Started server process [10] INFO: Waiting for application startup. INFO: Application startup complete. INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit) LOG: Connected to Hazelcast cluster LOG: Received transaction 1774013298945 INFO: 172.18.0.11:52564 - "POST /log HTTP/1.1" 200 OK LOG: Received transaction 1774013332487 INFO: 172.18.0.11:49706 - "POST /log HTTP/1.1" 200 OK LOG: Received transaction 1774013459684 INFO: 172.18.0.11:52386 - "POST /log HTTP/1.1" 200 OK LOG: Received transaction 1774013520223 INFO: 172.18.0.11:55142 - "POST /log HTTP/1.1" 200 OK INFO: 172.18.0.11:59168 - "GET /logs/1234 HTTP/1.1" 200 OK INFO: Shutting down INFO: Waiting for application shutdown. INFO: Application shutdown complete. INFO: Finished server process [10]</pre>					

Logs	Inspect	Bind mounts	Exec	Files	Stats
<pre>INFO: Started server process [10] INFO: Waiting for application startup. INFO: Application startup complete. INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit) INFO: 151.101.0.223:44824 - "POST /transaction HTTP/1.1" 200 OK INFO: 151.101.0.223:23249 - "POST /transaction HTTP/1.1" 200 OK INFO: 151.101.0.223:34633 - "POST /transaction HTTP/1.1" 200 OK INFO: 151.101.0.223:39883 - "POST /transaction HTTP/1.1" 200 OK INFO: 151.101.0.223:30058 - "POST /transaction HTTP/1.1" 200 OK INFO: 151.101.0.223:19865 - "POST /transaction HTTP/1.1" 200 OK INFO: 151.101.0.223:62716 - "POST /transaction HTTP/1.1" 200 OK INFO: 151.101.0.223:46854 - "POST /transaction HTTP/1.1" 200 OK INFO: 151.101.0.223:48214 - "POST /transaction HTTP/1.1" 200 OK INFO: 151.101.0.223:37634 - "POST /transaction HTTP/1.1" 200 OK INFO: 151.101.0.223:20477 - "GET /user/1234 HTTP/1.1" 200 OK INFO: 151.101.0.223:20845 - "POST /transaction HTTP/1.1" 422 Unprocessable Content INFO: 151.101.0.223:31080 - "POST /transaction HTTP/1.1" 200 OK INFO: 151.101.0.223:32780 - "POST /transaction HTTP/1.1" 200 OK INFO: 151.101.0.223:32780 - "POST /transaction HTTP/1.1" 200 OK FACADE: http://logging-service-3:8000 is down, trying another... INFO: 151.101.0.223:56181 - "POST /transaction HTTP/1.1" 200 OK INFO: 151.101.0.223:27350 - "GET /user/ira HTTP/1.1" 200 OK</pre>					

Я зробила 4 транзакції, четверта з них попала на третій logging-service, що не працює. Але після цього facade-service звернувся до іншого logging-service і операція виконалася успішно. На рахунку правильний сума - 400.



```

FACADE: http://logging-service-3:8000 is down, trying another...
INFO: 151.101.0.223:32564 - "POST /transaction HTTP/1.1" 200 OK
FACADE: http://logging-service-3:8000 is down, trying another...
INFO: 151.101.0.223:32564 - "POST /transaction HTTP/1.1" 200 OK
FACADE: http://logging-service-3:8000 is down, trying another...
FACADE: http://logging-service-1:8000 is down, trying another...
INFO: 151.101.0.223:32564 - "POST /transaction HTTP/1.1" 200 OK
INFO: 151.101.0.223:36398 - "POST /transaction HTTP/1.1" 200 OK
INFO: 151.101.0.223:30491 - "GET /user/ira HTTP/1.1" 200 OK

```

```
{
  "balance": 800,
  "transactions": [
    {
      "transaction_id": "1774014861374",
      "user_id": "Ira",
      "amount": 100
    },
    {
      "transaction_id": "1774014332269",
      "user_id": "Ira",
      "amount": 100
    },
    {
      "transaction_id": "1774014859186",
      "user_id": "Ira",
      "amount": 100
    },
    {
      "transaction_id": "1774014867300",
      "user_id": "Ira",
      "amount": 100
    }
  ],
}
```

Тут я зробила ще 4 транзакції, на першій і другій транзакції facade зміг звернутись до logging-service-2 з другого разу, а на третій лише з третього. На четвертій одразу з першого разу. Але всі транзакції успішні і загальна сума 800.

- **Тест 2.** Вимкнути один/два ноди Hazelcast та перевірити чи зможемо записати та прочитати транзакції
- **docker stop software architecture-hz-node-1-1**

ACTIVE

```
2026-03-20 14:00:10,928 [ INFO] [hz.ShutdownThread] [c.h.c.LifecycleService]: [hz-node-1]:5701 [log] [5.3.8] [hz-node-1]:5701 is SHUTTING DOWN
```

```
curl -X 'POST' \
'http://localhost:8080/transaction' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "user_id": "ira",
  "amount": 200
}'

{
  "transaction_id": "1774015272245",
  "balance": 1000
}
```

Member ^	^ Entries	^ Gets	^ Puts	^ Removals	^ Sets	^ Entry M
hz-node-2:5701	11	0	8	0	0	
hz-node-3:5701	8	0	7	0	0	
TOTAL	19	0	15	0	0	

Member	Entries	Gets	Puts	Removals	Sets	Entry M
hz-node-2:5701	11	0	8	0	0	
hz-node-3:5701	9	0	8	0	0	
TOTAL	20	0	16	0	0	

```
curl -X 'GET' \
'http://localhost:8080/user/ira' \
-H 'accept: application/json'
```

```
{
  "balance": 1200,
  "transactions": [
    {
      "transaction_id": "1774015973855",
      "user_id": "ira",
      "amount": 200
    },
    {
      "transaction_id": "1774014859186",
      "user_id": "ira",
      "amount": 100
    }
  ]
}
```

```
INFO: 151.101.0.223:65117 - "POST /transaction HTTP/1.1" 200 OK
INFO: 151.101.0.223:43478 - "POST /transaction HTTP/1.1" 200 OK
INFO: 151.101.0.223:36089 - "GET /user/ira HTTP/1.1" 200 OK
```

Дані перерозподілилися на 2 живі ноди, наступні операції на POST та GET пройшли успішно.

docker stop software_architecture-hz-node-2-1

```
2026-03-20 14:17:02,437 [ INFO] [hz.ShutdownThread] [c.h.i.i.Node]: [hz-node-2]:5701 [log] [5.3.8] Running shutdown hook... Current node state: ACTIVE
2026-03-20 14:17:02,441 [ INFO] [hz.ShutdownThread] [c.h.c.LifecycleService]: [hz-node-2]:5701 [log] [5.3.8] [hz-node-2]:5701 is SHUTTING_DOWN
```

Member	Entries	Gets	Puts	Removals	Sets	Entry M
hz-node-2:5701	N/A	N/A	N/A	N/A	N/A	
hz-node-3:5701	20	0	8	0	0	
TOTAL	20	0	8	0	0	

Тепер всі дані на одній ноді.

```
INFO: 151.101.0.223:33034 - "POST /transaction HTTP/1.1" 200 OK
INFO: 151.101.0.223:61491 - "GET /user/ira HTTP/1.1" 200 OK
```

```
curl -X 'GET' \
'http://localhost:8080/user/ira' \
-H 'accept: application/json'
```

```
{
  "balance": 1400,
  "transactions": [
    {
      "transaction_id": "1774015973855",
      "user_id": "ira",
      "amount": 200
    },
    {
      "transaction_id": "1774014861374",
      "user_id": "ira",
      "amount": 100
    }
  ]
}
```

І POST і GET запити пройшли успішно.

```
curl -X 'POST' \
'http://localhost:8080/transaction' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "transaction_id": "1774016356870",
  "user_id": "ira",
  "amount": 200
}'
```

```
{
  "transaction_id": "1774016356870",
  "balance": 1400
}
```

6. Тестування продуктивності, порівняння з результатами першого завдання
Використовувала ідентичний скрипт як в завданні 1.

1. 10 клієнтів одночасно роблять по 10К однакових транзакцій по додаванню 1 на свій рахунок. У результаті кінцеве значення балансу на 10-и рахунках має бути по 10К.

```
python3 test.py
(logging-service) ira@MacBook-Air-Ira software_architecture % python3 test.py

Resetting system...
10 clients 10 accounts scenario
Total time: 344.2643079169793
Requests/sec: 290.47449212805236

Service contribution:
Logging total time: 1565.1196989659193
Counter total time: 2423.5039285112416
Request count: 100000

Final balances:
{'user_5': 10000.0, 'user_3': 10000.0, 'user_4': 10000.0, 'user_2': 10000.0, 'user_7': 10000.0, 'user_9': 10000.0, 'user_6': 10000.0, 'user_0': 10000.0, 'user_1': 10000.0, 'user_8': 10000.0}
```

Завдання 3

```

Resetting system...
10 clients 10 accounts scenario
Total time: 215.4941074170638
Requests/sec: 464.04981184224044

Service contribution:
Logging total time: 1272.5887620283738
Counter total time: 1255.0846905879062
Request count: 100000

Final balances:
{'user_0': 10000, 'user_1': 10000, 'user_3': 10000, 'user_5': 10000, 'user_4': 10000, 'user_2': 10000, 'user_7': 10000, 'user_6': 10000, 'user_8': 10000, 'user_9': 10000}

```

Завдання 1

Результати для 3 завдання так само правильні, сумарна кількість запитів 100000, на кожному з 10 рахунків 10000. Порівнянню з завданням 1, весь час збільшився. Загальний зріс з 215с до 344с (майже в 1.5 раза). Причина: мережевих запитів стало значно більше. До того ж, замість 3-4 контейнерів тепер їх близько **10** (Фасад, 3 Логери, 3 Hazelcast, Counter, DB, ManCenter). Процесору треба набагато частіше робити **Context Switching**. Відповідно пропускна здатність впала з 464 req/sec до 290 req/sec. Раніше logging-service мав один екземпляр, а дані зберігалися в локальному списку RAM. Тепер ж їх 3 + Hazelcast In-Memory Map. Час логування зріс ~1565с проти ~1272с. Однією з причин ще є реплікація. Нода 1 одночасно відправляє ці дані на дві інші ноди (бекапи). Логер чекає підтвердження, що дані успішно скопійовані на всі 3 точки, перш ніж відповісти Фасаду «ОК».

Counter Service так само зберігав все в пам'яті і при docker-compose down всі дані зникали. Тепер я використовую PostgreSQL, час обробки зріс сильно ~2423с проти ~1255с. Це тому що запис на диск і робота з транзакціями бази даних - це найповільніша частина системи. Але тепер баланси персистентні.

2. 10 клієнтів одночасно роблять по 10К однакових транзакцій по додаванню 1 на один і той самий рахунок. У результаті кінцеве значення балансу на одному рахунках має бути 100K.

```

Resetting system...
10 clients 1 account scenario
Total time: 390.16588570800377
Requests/sec: 256.30123919864945

Service contribution:
Logging total time: 2021.2452928490538
Counter total time: 2771.666493508148
Request count: 100000

Final balances:
{'shared_user': 100000.0}

Resetting system...

```

Завдання 3

```

Resetting system...
10 clients 1 account scenario
Total time: 232.74651308299508
Requests/sec: 429.6519792085607

Service contribution:
Logging total time: 1371.2304886489856
Counter total time: 1358.6068351417089
Request count: 100000

Final balances:
{'shared_user': 100000}

```

Завдання 1

Це кейс, де 10 клієнтів пишуть на один рахунок. Результати теє повністю коректні, спільний рахунок має баланс 100000. Клієнти не переписували дані одне одного, бо SQL запит містить ON CONFLICT DO UPDATE, тобто рядок блокується на час оновлення, щоб лише один клієнт міг робити зміни.

Total time, як і в першому завданні, виріс(з 344 до 390). Відбувається це через блокування.

Для logging-service навантаження змінилося не критично порівняно з попереднім тестом (час ~2021с), бо логеру байдуже, чи це один юзер, чи різні - він просто записує нові ключі в Hazelcast.

A counter-service - вузьке місцем (час ~2771с). Весь приріст часу припадає саме на нього, бо Postgres змушений вистроювати 100 000 запитів у чергу до одного єдиного рядка в таблиці.